고가용성 가상 암호화폐 거래소 운영 설계

한국폴리텍대학 광명융합기술교육원 데이터분석과 파이널 프로젝트 계획서입니다. 1인 프로젝트입니다.

작성 2026-09-09. 최종 발표 2026-10-16.

이 문서는 저장소 실물에서 다시 유도해 새로 썼습니다. 종전 판(`docs/archive/기획안_원본_20260908.md`)은 폐기했습니다. 그 판은 2026-08-25 부터 여러 결정이 쌓이는 동안 갱신이 따라가지 못해, 저장소와 어긋난 주장이 여러 절에 남아 있었습니다.

이 문서의 수치는 전부 측정 출처를 함께 적었습니다. 측정으로 재현되지 않은 값은 "미측정" 또는 "설계값"으로 표기했습니다.

요약

무엇을 만드는가

봇이 24시간 매매하는 완전 시뮬레이션 암호화폐 거래소입니다. 실거래와 실자산이 없고, 실행 중에 외부 API를 부르지 않습니다. 12개 마켓에서 봇 플릿이 스스로 호가를 만들고 체결을 냅니다.

무엇을 증명하는가

이 프로젝트가 답하려는 질문은 하나입니다.

> **24시간 도는 거래소에 트래픽이 몰리거나 고장이 나면, 어느 계층이 먼저 터지는가. 그것을 어떻게 알아내고, 무엇을 키워서, 어떻게 안정화하는가.**

핵심은 병목이 한 곳에 고정되어 있지 않다는 점입니다. 한 계층을 키우면 부하가 다음 계층으로 옮겨 갑니다. 데이터베이스를 키우면 요청이 빨리 끝나고, 그만큼 애플리케이션이 더 많은 일을 하게 되어 이번에는 애플리케이션이 병목이 됩니다. 그래서 이 프로젝트는 "한 번 잘 설계한다"가 아니라 "관측하고, 병목을 찾고, 키우고, 옮겨간 병목을 다시 찾는" 반복을 다룹니다.

그 반복을 세 축으로 나눠 증명합니다.

축	증명하려는 것	반증 조건
무결	폭주와 장애 아래에서도 자산별 원장 합이 0이고, 주문 중복 접수와 중복 체결과 유실이 0입니다	불변식 27개 중 어느 하나라도 위반 행이 1행 나오면 반증입니다
복구	리더 노드를 죽여도 사람 손 없이 다른 노드가 승격해 쓰기가 재개됩니다	30초를 넘거나, 승격이 일어나지 않거나, 사람이 손대야 재개되면 반증입니다
안정화	병목 계층을 관측으로 특정하고, 그 계	병목을 특정하지 못하거나, 키웠는데 지

	층을 키웠을 때 지표가 예측한 방향으로 움직입니다	표가 움직이지 않으면 반증입니다

앞의 두 축은 종전 판에서 이어받았습니다. 세 번째 축이 이번에 추가되었습니다. 2026-09-08 실측이 그 축의 첫 구간을 잡았기 때문입니다.

지금까지 무엇이 나왔는가

가장 중요한 실측 셋입니다.

하나. 데이터베이스 규격을 키우면 지연이 절반 이하로 줄지만 처리량은 그대로입니다.

데이터베이스 컨테이너의 CPU 한도만 1코어에서 4코어로 바꿔 세 쌍을 측정했습니다. 응답 시간 상위 1퍼센트(p99)가 318에서 121밀리초, 382에서 146밀리초, 416에서 127밀리초로 세 쌍 모두 60퍼센트 넘게 줄었습니다. 같은 회차에서 초당 체결 수는 16.9에서 17.7로 사실상 그대로였습니다. 근거는 `harness/evidence/pool-db-matrix-20260908-204634/`입니다.

둘. 커넥션 풀과 수용 정원을 함께 올리면 지연이 오히려 나빠집니다.

풀을 16에서 48로 올리면 수용 정원이 6에서 19로 따라 오릅니다. 세 쌍 모두 지연이 악화되었습니다(138에서 223, 86에서 108, 233에서 261밀리초). 커넥션 대기는 전 회차 0이었으므로 커넥션이 모자란 상태가 아니었고, 정원만 늘리면 데이터베이스 안에서 경합만 커집니다.

셋. 동기 복제 대상이 한 대뿐이면 그 한 대가 쓰기의 단일 장애점입니다.

동기 복제 대상 대기 서버를 한 대만 두고 그 한 대를 죽이면, 주문 접수가 초당 97.2건에서 2초 만에 0건이 됩니다. 두 대를 두고 정족수를 `ANY 1`로 묶으면 한 대가 죽어도 초당 56건으로 이어집니다. 근거는 `harness/evidence/replication-fault-20260908/`입니다.

비용과 일정

발표까지 총계 미국 달러 177.60(원화 248,640원)입니다. 상한은 달력 월 실청구로 판정하며, 2026년 9월과 10월이 각각 117,242원으로 자체 상한 500,000원 안입니다. 상한은 2026-09-08에 지도교수 승인으로 100,000원에서 올렸습니다.

지원 범위 안입니다. 기존 크레딧 잔액 163.71달러에 더해, 2026-09-08 지도교수 승인분 50만원 이상이 다음 주에 크레딧으로 지급될 예정입니다. 종전 판은 총계를 157.91달러로 적고 잔여 5.80달러가 남는다고 했는데, 그 계산이 확정 단가 소계를 153.66이 아니라 135.77로 잘못 쓴 데서 나왔습니다.

일정은 2026-09-01 착수, 10-09 기능 동결, 10-16 발표입니다. 09-08로 잡았던 외부 접속 성립 고정점은 달성하지 못했습니다. 클라우드에 인스턴스를 세운 적이 0건이고 도메인 위임도 끝나지 않았습니다.

1. 프로젝트 개요

1.1 한 줄 정의

봇 플릿이 실시간으로 매매하는 완전 시뮬레이션 암호화폐 거래소입니다. 실거래와 실자산과 런타임 외부 API 의존이 없습니다. 이 시스템에 장애를 주입하면서 무결과 복구와 안정화를 측정합니다.

"완전 시뮬레이션"이라는 말은 시장 데이터까지 내부에서 만든다는 뜻입니다. 실행 중에 외부 거래소 API를 부르지 않습니다. 그래야 외부 서비스의 장애가 우리 실험 결과에 섞이지 않습니다.

1.2 왜 거래소인가

거래소를 고른 것은 개발 취향이 아니라 실험 설계 판단입니다.

첫째, 복식부기 원장이 회계 불변식을 도메인 성질로 제공합니다. 복식부기란 모든 거래를 차변과 대변 두 쪽에 같은 금액으로 적는 방식입니다. 그래서 자산별로 모든 항목을 더하면 항상 0이어야 합니다. 이 0이라는 숫자가 판정 술어가 됩니다. 사람이 눈으로 보고 "정상인 것 같다"고 말하는 대신, SQL 한 줄이 위반 행 수를 돌려줍니다.

둘째, 주문 처리는 순서가 결과를 바꿉니다. 같은 주문 묶음도 처리 순서가 다르면 체결 결과가 달라집니다. 그래서 장애 뒤에 상태를 복원할 때 "순서까지 같게 복원했는가"를 물을 수 있습니다.

셋째, 24시간 도는 시스템이라 정지 자체가 사고입니다. 야간에 사람이 없어도 복구되어야 한다는 요구가 자연스럽게 나옵니다.

1.3 무엇을 증명하는가

기능 목록이 아니라 명제를 증명합니다. 명제는 반증 조건을 함께 적을 때만 명제입니다.

구분	증명 1 체결 무결	증명 2 자동 복구	증명 3 병목 이동과 안정화
주장	폭주와 장애 아래에서도 자산별 원장 합이 0이고 중복과 유실이 0입니다	리더를 죽여도 사람 손 없이 승격해 쓰기가 재개됩니다	병목 계층을 관측으로 특정하고, 키웠을 때 지표가 예측대로 움직입니다
관측 대상	`ledger_entries`, `balances`, `command_journal`, `event_journal`, `trades`	`leader_lease.epoch`, `command_journal.fencing_token`, `trades.engine_ts`	계층별 CPU, 응답 시간 분위수, 수용 정원 점유, 커넥션 대기
판정 술어	불변식 27개의 위반 행 수	승격 후 최초 신규 체결의 엔진 시각에서 정지 시각을 뺀 값	조작 전후 지표 차이와 그 방향
통과 기준	전 항목 0행	30초 이내, 세대 번호 정확히 1 증가, 수동 개입 0회	예측한 방향으로 유의하게 움직이고 재현됩니다
반증 조건	어느 술어라도 1행 반환	30초 초과, 승격 미발생, 사람 개입 필요	병목 미특정, 또는 조작이 지표를 움직이지 못함

세 번째 축의 "예측한 방향"이란 이런 뜻입니다. 데이터베이스가 병목이라고 판정했으면, 데이터베이스를 키웠을 때 응답 시간이 줄어야 합니다. 줄지 않으면 병목 판정이 틀린 것입니다.

1.4 무엇을 하지 않는가

- 실제 화폐와 실제 자산을 다루지 않습니다.
- 실행 중에 외부 거래소 API를 부르지 않습니다. 초기 시세 데이터는 빌드 시점에 고정한 값입니다.
- 사람이 눈으로 보고 판정하지 않습니다. 모든 판정은 SQL 위반 행 수, 지표 임계, 종료 코드 중 하나입니다.

2. 문제 정의

2.1 24시간 거래소에서 무엇이 터지는가

거래소는 멈추면 안 되는 시스템입니다. 그런데 멈추게 만드는 원인은 하나가 아닙니다. 이 프로젝트는 원인을 세 갈래로 나눕니다.

갈래 하나. 트래픽이 몰릴 때.

주문이 평소보다 훨씬 많이 들어오면 어딘가가 먼저 한계에 닿습니다. 그 "어딘가"가 어디인지는 미리 알 수 없습니다. 데이터베이스일 수도 있고, 애플리케이션의 CPU일 수도 있고, 커넥션 풀일 수도 있고, 수용 정원일 수도 있습니다. 관측으로 특정해야 합니다.

갈래 둘. 부품이 고장 날 때.

노드 하나가 죽거나, 네트워크가 끊기거나, 디스크가 느려집니다. 이때 시스템 전체가 멈추는지, 아니면 남은 부품으로 계속 도는지가 갈립니다. 그리고 고장이 끝난 뒤 스스로 돌아오는지도 갈립니다.

갈래 셋. 고장이 아닌데 느려질 때.

프로세스가 죽지 않고 살아 있는데 응답만 안 하는 상태가 있습니다. 이것을 회색 장애라고 부릅니다. 죽은 것보다 다루기 어렵습니다. 감시 장치가 "살아 있음"만 보면 정상으로 판정하기 때문입니다.

2.2 왜 병목이 이동하는가

한 계층을 키우면 그 계층이 더 이상 병목이 아니게 됩니다. 그런데 그 순간 부하가 다음 계층으로 옮겨 갑니다.

구체적으로 이렇습니다. 데이터베이스가 느리면 애플리케이션은 데이터베이스 응답을 기다리며 놉니다. 이때 애플리케이션 CPU는 낮습니다. 데이터베이스를 키우면 응답이 빨라지고, 애플리케이션은 기다리는 시간이 줄어 더 많은 요청을 처리합니다. 그러면 이번에는 애플리케이션 CPU가 올라갑니다.

그래서 "데이터베이스를 키웠더니 애플리케이션이 터졌다"는 일이 실제로 일어납니다. 그것은 실패가 아니라 병목이 이동한 것입니다. 다음 조작은 애플리케이션 노드를 늘리는 것입니다.

이 이동을 눈으로 보려면 두 가지가 필요합니다. 계층별 지표를 동시에 보는 관측과, 한 번에 한 축만 바꾸는 실험 설계입니다.

2.3 왜 이것을 증명이라고 부르는가

"고가용성 시스템을 만들었습니다"라는 문장은 검증할 수 없습니다. 무엇이 참이면 그 문장이 거짓이 되는지가 없기 때문입니다.

그래서 이 프로젝트는 모든 주장에 반증 조건을 붙입니다. 원장 합이 0이 아니면 무결이 깨진 것입니다. 30초 안에 복구되지 않으면 자동 복구가 깨진 것입니다. 데이터베이스를 키웠는데 응답 시간이 안 줄면 병목 판정이 틀린 것입니다.

판정은 사람이 하지 않습니다. SQL이 위반 행 수를 돌려주고, 스크립트가 종료 코드를 돌려줍니다.

3. 현재 진척

3.1 성립한 것

아래는 저장소에 실물이 있고 측정으로 확인한 항목입니다.

축	상태	근거
매칭엔진	성립. 단일 라이터 구조, 결정론적 재생	`server/app/src/main/java/kr/exchange/engine/`
복식부기 원장	성립. 불변식 27개를 SQL로 판정	`harness/sql/invariants.sql`
리더 선출과 세대 번호 펜싱	성립. 세대 번호가 엄격히 증가하고 구 리더의 쓰기가 거부됨	`harness/evidence/leader-failover-20260908/`
리더 임차를 커밋 경로 밖으로 분리	성립. 다만 선택 경로이고 기본값이 아님	ADR 0017, `harness/evidence/leader-lease-dynamodb-20260908/`
부하 중 노드 재합류	성립. 재합류 20.8초, 그 창의 접수가 정상 구간과 같음	`harness/evidence/follower-rejoin-20260908/`
동기 복제 정족수	성립. 한 대 상실을 견딤	ADR 0018, `harness/evidence/replication-fault-20260908/`
수용 제한	성립. 커넥션 풀에서 정원을 유도하고 초과를 거절로 막음	`server/app/src/main/java/kr/exchange/api/common/AdmissionLimits.java`
카오스 주입	성립. 스크립트 9종	`harness/scripts/` 의 e1 부터 e9
관측 자산	성립. 레코딩 규칙 9개, 알람 15개, 대시보드 9종	`harness/prometheus/rules/`, `harness/grafana/dashboards/`
통합 검증	성립. 검증 클래스 74개가 전량 통과	`server/app/src/test/`

3.2 성립하지 않은 것

정직하게 적습니다. 문서에 설계가 있다는 이유로 완료로 적지 않습니다.

축	상태	무엇이 없는가
클라우드 배포	미성립	AWS에 배포한 적이 없습니다. `.tf` 파일이 0개입니다

상시면 관측 배선	미성립	배포용 컴포즈 3종에 관측 서비스 정의가 0건입니다. 관측 스택은 로컬 카오스 스택에서만 돕니다
계층별 프로브	미성립	네트워크 계층과 전송 계층과 응용 계층을 각각 찌르는 프로브가 배선되지 않았습니다
알림 실투입	미성립	알림 수신 주소를 넣지 않았습니다. 로컬에서 도달만 확인했습니다
커널 계층 장애 주입	미성립	실험 E10이 미실시입니다. 종전 E1부터 E9까지는 전부 프록시 모사입니다
승격 시 동기 대상 재설정	미성립	승격한 노드를 기준으로 동기 복제 대상을 다시 쓰는 절차가 배선되지 않았습니다

3.3 측정 평면의 한계

지금까지의 실측은 전부 로컬 도커 환경입니다. 배포 대상 인스턴스 실물에서 잰 값이 0건입니다.

그리고 같은 기계에서 여러 컨테이너가 함께 돌았습니다. 2026-09-08 회차에서는 스택 12개가 상시 돌았고, 통합 검증이 겹친 구간에는 임시 데이터베이스 컨테이너가 최대 27개까지 함께 있었습니다. 그래서 이 문서의 성능 수치는 절대값이 아니라 같은 주행 안의 상대 비교로 읽어야 합니다.

4. 기술 선택

4.1 선택의 원칙

기술을 고를 때 기준이 하나입니다. **장애를 주입할 권한이 없는 계층은 증명 대상에서 빠집니다.**

이 프로젝트가 증명하려는 것이 장애 아래의 무결과 복구인데, 그 장애를 만들 수 없으면 증명할 것이 없습니다. 그래서 편한 관리형 서비스 대신 직접 운영하는 쪽을 여러 번 골랐습니다. 대가는 정직하게 계상합니다.

4.2 선택표

축	선택	사유
언어와 프레임워크	Java 21, Spring Boot	가상 스레드로 봇 수천 개를 한 프로세스에서 돌립니다
데이터베이스	EC2에 직접 운영하는 PostgreSQL 17	관리형 데이터베이스는 운영체제 접근이 없어 커널 파라미터 조작과 프로세스 정지를 못 합니다. 그러면 데이터베이스 계층이 증명 범위에서 빠집니다 (ADR 0004)
컴퓨트	EC2 Graviton (t4g, c7g)	운영체제 셸 접근이 필요합니다. Graviton은 x86 대비 상시 구동에서 약

		20퍼센트 저렴합니다
로드밸런서	Application Load Balancer, 2 가용영역	두 가용영역의 진입점을 이중화하려면 관리형이 필요합니다. 헬스체크가 애플리케이션의 상태 확인 경로를 그대로 읽으므로 판정 주체가 애플리케이션이 됩니다
확장	EC2 Auto Scaling 그룹. 확장 정책은 두지 않음	조회 계층에만 겁니다. 최소 2, 희망 2, 최대 3입니다. 확장 방식은 수동입니다. 자동 확장만 관리형 감시 서비스를 요구하는데 그 서비스를 비용으로 기각했으므로, 정책 없이 희망 용량만 직접 설정합니다 (ADR 0015)
리더 임차	임차 보관소를 갈아 끼울 수 있게 두고 구현을 둘 둡니다. 데이터베이스 행과 DynamoDB 항목 1개입니다	임차 갱신이 데이터베이스 커밋이면, 동기 복제 대상을 전부 잃을 때 갱신도 함께 멈춥니다. 그러면 임차가 만료되고 복제가 돌아와도 애플리케이션 재기동 없이는 복구되지 않습니다. DynamoDB 구현이 그 결합을 끊습니다. 기본값은 데이터베이스 쪽으로 두었습니다. 옮기기 전과 옮긴 뒤를 같은 조건에서 재려면 종전 경로가 살아 있어야 하기 때문입니다 (ADR 0017)
관측	Prometheus와 Grafana 직접 운영	레코딩 규칙 9개와 알람 15개와 대시보드 9종이 이미 파일로 있고 지속적 통합이 검증합니다. 관리형 감시 서비스로 옮기면 월 약 190달러가 듭니다
네트워크 주소 변환	t4g.nano 인스턴스 2대, 가용영역별로 하나씩	관리형 게이트웨이 대비 월 78.55달러를 아낍니다. 동시에 방화벽 규칙 실물 근거가 됩니다
정적 웹	S3와 CloudFront	애플리케이션 노드 자원을 쓰지 않고, 원본 장애와 웹 가용성을 분리합니다
시크릿	SSM 파라미터 스토어	시크릿당 월 0.40달러를 아낍니다
관리자 접근	SSM 세션 매니저	원격 접속 포트를 아예 열지 않습니다. 세션 기록이 그 자체로 감사 증거입니다
지속적 통합 인증	GitHub Actions OIDC 페더레이션	장기 접근 키를 발급하지 않습니다
오케스트레이터 (클라우드)	채택하지 않음	스케줄러가 컨테이너를 자동 재기동하면 노드를 죽이는 실험의 관측이 오염됩니다
인프라 코드화	Terraform. 여유 시간 조건부	아직 착수하지 않았습니다

4.3 비채택과 그 사유

비채택	사유

관리형 관계형 데이터베이스	운영체제 접근이 없어 데이터베이스 계층 장애 주입이 불가능합니다
관리형 네트워크 주소 변환 게이트웨이	월 78.55달러 차이입니다
관리형 감시 서비스	월 약 190달러입니다. 그리고 자동 확장 정책이 이 서비스를 요구하므로, 비채택이 곧 수동 확장 채택으로 이어집니다
관리형 쿠버네티스	컨트롤플레인 자체가 시간 과금입니다
상시 사설망 터널	두 평면을 잇는 것은 네트워크가 아니라 같은 이미지 지문과 저장소에 커밋한 판정 파일입니다
시크릿 매니저	표준 파라미터로 충분합니다

5. 평면 분리

5.1 왜 평면을 나누는가

학교 필수 조건인 외부 상시 접속과, 이 프로젝트가 필요로 하는 커널 계층 장애 주입은 한 평면에서 양립하지 않습니다. 커널에 장애를 주입하면 그 노드에 접근하는 경로도 함께 끊길 수 있기 때문입니다.

그래서 평면을 셋으로 나눕니다.

평면	위치	구동	허용하는 실험
상시면	AWS 서울 리전	24시간	애플리케이션 계층 정지와 가용영역 상실 재현. 커널 주입은 제외합니다
증명면	AWS, 창 구동	실험과 발표 창	부하 주입과 확정 회차 재현. 창이 끝나면 볼륨까지 파기합니다
검증면	온프레미스, 세션 구동	실험 세션 중	커널 계층 주입 전량. 비용이 0이라 회차 수에 제약이 없습니다

5.2 배치 규칙

> 상시 필요한 것은 AWS에 둡니다. 세션 중에만 필요한 것은 검증면에 둡니다.

이 규칙이 관측 배치를 정합니다. Prometheus는 대상에게 가서 지표를 가져오는 방식입니다. 가져오기를 놓치면 그 구간의 시계열이 영구 공백이 됩니다. 나중에 채울 수단이 없습니다. 그래서 판정에 걸린 관측은 대상과 같은 평면인 AWS에 둡니다.

5.3 두 평면을 잇는 것

상시 터널을 두지 않습니다. 두 평면을 잇는 것은 네트워크가 아니라 둘입니다.

- 같은 컨테이너 이미지 지문. 지속적 통합이 이미지를 만들어 레지스트리에 올리고, 양쪽이 같은 지문을 받습니다.
- 저장소에 커밋한 판정 파일. 검증면에서 나온 실험 결과를 저장소에 넣고, 그것이 증거가 됩니다.

검증면 기계가 상시 구동이 아니므로 상시 터널을 전제하면 그 전제가 먼저 깨집니다.

6. 인프라와 배포

6.1 전체 구조

사용자 요청은 도메인 이름 해석을 거쳐 콘텐츠 전송 네트워크로 들어옵니다. 정적 화면은 오브젝트 스토리지에서 바로 나가고, API와 실시간 연결은 로드밸런서로 넘어갑니다.

로드밸런서 뒤에는 애플리케이션 노드가 있고, 그 뒤에 데이터베이스가 있습니다. 리더 임차는 별도 저장소로 뺄 수 있게 해 두었습니다.

6.2 노드 구성

노드	등급	가용영역	구동	역할
app-1	t4g.small	a	상시	리더. 매칭엔진이 여기서만 돕니다
app-2	t4g.small	c	상시	조회를 받습니다. 리더가 죽으면 승격 대상입니다
app-3	t4g.small	c	증설분	평시에는 없습니다. 관리자가 버튼으로 늘립니다
pg-primary	t4g.small	a	상시	쓰기를 받습니다
pg-standby-1	t4g.small	c	상시	동기 복제 대상입니다
pg-standby-2	t4g.small	c	상시	동기 복제 대상입니다. 정족수의 두 번째 표입니다
bot-1	t4g.small	c	상시	봇 플릿과 관측 스택이 함께 돕니다
NAT-a	t4g.nano	a	상시	사설 서브넷의 외부 접속 경로입니다
NAT-c	t4g.nano	c	상시	같습니다. 가용영역별로 둡니다
load	c7g.large	창 구동	실험 창	부하 발생기 전용입니다. 버스트가 없는 등급이라 신용 소진이 측정에 섞이지 않습니다

봇과 관측을 같은 노드에 둔 이유는 둘입니다. 감시는 피검체 밖에 있어야 성립하고, 가용영역 a를 잃었을 때 승격을 관측할 수 있어야 합니다 (ADR 0016).

6.3 동기 복제와 정족수

데이터베이스는 primary 한 대와 대기 서버 두 대입니다. 동기 복제 대상을 `ANY 1`로 묶습니다. 커밋할 때 두 대기 서버 중 아무 한 대의 응답만 기다린다는 뜻입니다.

대기 서버를 한 대만 두면 그 한 대가 쓰기의 단일 장애점이 됩니다. 2026-09-08 실측이 그것입니다.

조건	초당 접수	서버 오류	커밋
동기 대기 1대 정상	97.2	0	정상
동기 대기 1대 정지	0.0	0	전면 정지
정족수, 2대 중 1대 정지	56.0	0	정상 지속
정족수, 2대 모두 정지	0.0	0	전면 정지

배치는 primary가 가용영역 a, 대기 서버 둘 다 가용영역 c입니다. 세 노드를 두 가용영역에 나누면 한쪽이 반드시 두 노드를 갖고, 그쪽을 잃으면 남은 한 대가 정족수를 못 채워 정지합니다. 어느 배치를 골라도 한쪽만 견딥니다. 이 프로젝트가 증명하려는 경로가 primary가 있는 가용영역의 상실이므로 그쪽을 견디는 배치를 골랐습니다 (ADR 0018).

6.4 조회 라우팅

로드밸런서 뒤에 대상 그룹이 둘입니다.

대상 그룹	대상	트래픽 포트	헬스체크 경로
쓰기	리더 1대	8080	준비 상태 경로
조회	리더와 조회 노드 전부. 2대에서 3대 가변	8080	읽기 전용 상태 경로

두 대상 그룹의 트래픽 포트는 둘 다 8080이고 헬스체크 포트도 둘 다 8081입니다. 갈리는 것은 헬스체크 경로입니다.

조회 대상에 리더가 들어갑니다. 리더를 빼면 조회 분산이 구조적으로 0이 되기 때문입니다. 팔로워는 준비 상태 경로에서 거절을 내므로 쓰기 대상 그룹에서만 빠집니다.

6.5 리더 임차와 세대 번호

한 시점에 쓰기를 할 수 있는 노드는 하나입니다. 그 권한을 임차로 관리합니다.

임차 갱신은 조건부 쓰기로 합니다. 초당 한 번입니다.

보관소는 갈아 끼울 수 있습니다. 구현이 둘입니다. 데이터베이스 행을 쓰는 것과 DynamoDB 항목 하나를 쓰는 것입니다. 설정 한 줄로 고릅니다.

현재 기본값은 데이터베이스 쪽입니다. 옮기기 전과 옮긴 뒤를 같은 조건에서 재려면 종전 경로가 살아 있어야 하기 때문입니다. DynamoDB 쪽을 고르면 갱신이 데이터베이스 커밋 경로 밖으로 나갑니다. 아래에 적은 실측이 그 경로의 것입니다 (ADR 0017).

세대 번호 검증은 데이터베이스에 남습니다. 쓰기 트랜잭션 안에서 "지금 이 노드가 현재 세대의 리더인가"를 확인합니다. 구 리더가 살아 돌아와 쓰려고 하면 그 검증에서 막힙니다.

임차 갱신을 데이터베이스 밖으로 옮긴 이유는 실측에서 나왔습니다. 갱신 자체가 커밋이면 동기 대기 서버를 전부 잃을 때 갱신도 함께 멈춥니다. 그러면 임차가 만료되는데 그것을 확인할 커밋이 막혀 있어 아무도 모르고, 복제가 돌아와 커밋이 풀리는 순간 세대 번호 검증이 리더를 끌어내립니다. 옮긴 뒤 같은 장애를 다시 내니 87초 정지 동안 갱신이 초당 0.35회로 계속되었고 애플리케이션 재기동 없이 3초 만에 정상화되었습니다 (ADR 0017).

6.6 웹 배치

웹은 빌드 시점에 정적 파일로 굽습니다. 실행 시점에 서버가 필요 없습니다.

API 주소는 빈 문자열입니다. 같은 출처로 부른다는 뜻입니다. 콘텐츠 전송 네트워크가 경로로 갈라, 정적 파일은 오브젝트 스토리지에서 API는 로드밸런서에서 가져옵니다. 그래서 브라우저 입장에서는 전부 같은 주소이고 교차 출처 문제가 없습니다.

7. 운영. 관측에서 안정화까지

이 절이 세 번째 증명 축을 담습니다. 부하가 몰리거나 고장이 났을 때 무엇을 보고 무엇을 하는지를 순서로 적습니다.

7.1 순환 구조

운영은 네 단계의 반복입니다.

```
관측 -> 병목 판정 -> 조작 -> 재관측
  ^                          |
  +--------------------------+
```

한 바퀴가 끝나면 병목이 다른 계층으로 옮겨가 있습니다. 그래서 같은 절차를 다시 돕니다. 이 반복이 안정화입니다.

7.2 무엇을 보는가

계층별로 지표를 나눠 봅니다. 한 계층만 보면 병목을 특정할 수 없습니다.

계층	지표	이 지표가 높으면
진입	로드밸런서의 노드별 응답 시간과 오류율	특정 노드만 나쁘면 그 노드 문제, 전부 나쁘면 하류 문제입니다
애플리케이션	프로세스 CPU, 응답 시간 분위수, 수용 정원 점유	CPU가 높으면 노드를 늘립니다

수용 제한	거절 수, 정원 점유율	정원이 꽉 차 거절이 나면 상류에서 막고 있는 것입니다
커넥션	사용 중 수, 대기 수, 확보 시간	대기가 있으면 커넥션이 모자란 것입니다
데이터베이스	CPU, 커밋 지연, 복제 지연, 잠금	CPU가 한도에 붙으면 규격을 올립니다

관측 자산은 레코딩 규칙 9개, 알람 15개, 대시보드 9종입니다. 지속적 통합이 규칙 문법을 검사합니다.

7.3 어느 계층이 병목인지 어떻게 아는가

지표 하나만 보면 오판합니다. 조합으로 봅니다.

관측	병목 판정
데이터베이스 CPU가 한도에 붙고, 애플리케이션 CPU는 낮고, 커넥션 대기가 있다	데이터베이스가 병목입니다
애플리케이션 CPU가 높고, 데이터베이스 CPU는 여유가 있다	애플리케이션이 병목입니다. 노드를 늘립니다
양쪽 CPU가 다 낮은데 거절이 난다	수용 정원이 병목입니다
양쪽 CPU가 다 낮고 커넥션 대기도 없는데 느리다	저장소 지연일 가능성이 큽니다
커넥션 대기가 있는데 데이터베이스 CPU가 낮다	커넥션 풀 크기가 병목입니다

2026-09-08 실측에서 마지막 두 줄을 구분하는 값이 나왔습니다. 커넥션 대기가 전 회차 0이었고 데이터베이스가 자기 한도의 절반도 안 쓰고 있었습니다. 데이터베이스 대기 사유를 열어 보니 38개 커넥션 중 35개가 "클라이언트의 다음 요청을 기다리는 중"이었습니다. 밀려서 막힌 것이 아니라 일이 없어서 노는 상태였습니다.

7.4 무엇을 조작하는가

계층마다 조작 수단이 다릅니다.

계층	조작	방식	효과
조회 애플리케이션	노드 수 증감	관리 화면 버튼	조회 용량이 늘어납니다
쓰기 애플리케이션	없음		단일 라이터라 노드를 늘려도 쓰기 천장이 안 오릅니다
데이터베이스	규격 상향	인스턴스 교체	응답 시간이 줄어듭니다
커넥션 풀과 정원	크기 조정	설정	실측에서는 지연이 악화되었습니다

조회 노드는 버튼이고 데이터베이스는 권고입니다. 조회 노드는 늘리면 바로 용량이 늘어 버튼으로 둘 값이 있습니다. 데이터베이스는 늘려도 쓰기 천장이 안 오르고 규격 상향은 인스턴스 교체라 버튼으로 누를 일이 아닙니다.

7.5 스케일링 실행 주체

판정은 자동이고 실행은 사람입니다 (ADR 0015).

Prometheus가 노드 지표를 모으고 임계를 넘으면 관리 화면에 "부하 높음, 증설 권장"을 표시합니다. 관리자가 버튼을 누르면 요청이 관리 API를 지나 클라우드 API를 부르고 조회 노드가 한 대 늡니다.

실행을 사람에게 둔 이유가 셋입니다.

첫째, 스케줄러가 컨테이너를 자동 재기동하면 노드를 죽이는 실험의 관측이 오염됩니다. 복구 시작 시점을 사람이 정하면 관측 창을 사람이 소유합니다.

둘째, CPU 하나로는 부하와 정지를 구분하지 못합니다. 정지한 노드의 CPU를 실측했더니 1.00에서 1.05코어였고, 이는 임계 근처입니다.

셋째, 증설이 답이 아닌 축이 있습니다. 주문은 단일 라이터라 노드를 늘려도 오르지 않습니다. 어느 축이 포화인지 가르는 것은 판별 절차이고 확장기가 대신하지 못합니다.

7.6 데이터베이스 규격 상향

데이터베이스는 노드 수가 아니라 규격을 바꿉니다. 확장 대상이 조회 계층뿐이라는 규정은 노드 수 증감과 그 버튼화를 배제한 것이지 규격 상향까지 배제한 것이 아닙니다.

2026-09-08 실측에서 규격 상향의 값이 나왔습니다. 데이터베이스 CPU 한도만 1코어에서 4코어로 바꿨습니다.

조건	응답 시간 상위 5퍼센트	상위 1퍼센트
풀 16, 40 OPS	138에서 86밀리초	318에서 121밀리초
풀 48, 40 OPS	223에서 108밀리초	382에서 146밀리초
풀 16, 고부하 회차	233에서 96밀리초	416에서 127밀리초

세 쌍 모두 상위 1퍼센트가 60퍼센트 넘게 줄었습니다. 꼬리가 먼저 줄어듭니다.

같은 회차에서 데이터베이스가 쓰는 CPU 양 자체는 줄지 않았습니다. 방향이 일관되지 않고 크기도 표본 산포 안이었습니다. 코어를 늘리는 것은 같은 일을 넓게 펴는 것이지 일을 줄이는 것이 아닙니다. 얻는 것은 대기 시간 제거입니다.

초당 체결 수는 16.9에서 17.7로 사실상 그대로였습니다. 수요를 천장 아래로 고정했기 때문입니다. 요청이 몰릴 때는 다릅니다. 요청 하나가 정원 한 칸을 붙들고 있는 시간이 3분의 1로 줄면 같은 정원으로 더 많이 통과시킬 수 있습니다.

7.7 무중단 규격 상향 절차

규격 상향을 서비스 중단 없이 하는 절차가 정족수 위에 섭니다. 대기 서버가 한 대뿐이면 그 한 대를 내리는 순간 커밋이 전면 정지하므로 이 절차 자체가 불가능합니다.

단계	조작	그동안의 쓰기
1	대기 서버 2를 큰 규격으로 교체	계속됩니다. 대기 서버 1이 정족수를 채웁니다
2	복제가 따라잡을 때까지 대기	계속됩니다
3	대기 서버 2로 승격	승격 창에서 멈춥니다
4	옛 primary를 큰 규격으로 교체해 대기 서버로 재합류	계속됩니다
5	동기 복제 대상을 새 primary 기준으로 재설정	계속됩니다

멈추는 구간은 3단계 하나뿐이고 그 길이는 승격 시간입니다. 5단계가 필요한 이유는 승격한 노드가 자기 자신을 동기 대상으로 셀 수 없기 때문입니다. 그 재설정은 아직 배선이 없습니다.

7.8 병목이 옮겨간 뒤

데이터베이스를 키우면 응답이 빨라지고 애플리케이션이 더 많은 요청을 처리합니다. 그러면 애플리케이션 CPU가 올라갑니다. 다음 조작은 조회 노드를 늘리는 것입니다.

이 이동은 아직 실측으로 확인하지 못했습니다. 수요를 천장 아래로 고정했기 때문에 데이터베이스가 빨라져도 애플리케이션이 더 할 일이 없었습니다. 고부하 회차를 시도했으나 실효 부하가 오르지 않았습니다. 무엇이 상한을 만들었는지는 아직 판정하지 못했습니다.

이것이 이 축의 남은 작업입니다.

7.9 백업과 복구

대상	방식	주기	상태
데이터베이스 물리 백업	블록 스토리지 스냅샷	일 1회	목표. 미배선
데이터베이스 논리 백업	기저 백업 도구	주 1회	목표. 미배선
미리 쓰기 기록 보관	오브젝트 스토리지로 전송	연속	목표. 미배선. 설정값이 자리 표시자입니다
복제	대기 서버 2대 동기 스트리밍	24시간	설계 확정. 클라우드 미배포
실험 증거	판정 파일을 저장소에 커밋	실험마다	성립. 30건

백업 항목은 전부 목표입니다. 배선한 것으로 적지 않습니다.

7.10 침묵 판정

체결이 0인 상태가 두 가지입니다. 정말 거래가 없는 것과, 무언가 고장 나서 주문이 안 들어오는 것입니다. 둘을 구분해야 합니다.

봇 플릿이 24시간 기저 트래픽을 만드는 것이 그 구분 수단입니다. 봇이 도는데 체결이 0이면 평온이 아니라 사고입니다.

다만 봇 하나에 판정을 걸지 않습니다. 봇은 내부 경로만 왕복하므로 로드밸런서와 콘텐츠 전송 네트워크 구간의 고장을 잡지 못합니다. 외부에서 찌르는 프로브가 그 구간을 맡아야 하는데 아직 배선하지 않았습니다.

8. 보안

8.1 접근 경로

대상	경로	통제
고객 평면	콘텐츠 전송 네트워크에서 로드밸런서	공개
관리 평면	별도 포트, 상호 인증서 검증	인증서 신원

하네스 API	상시면에 등록하지 않음	비활성
상태 확인 경로	내부에서만	외부 라우팅 없음
서버 셸	SSM 세션 매니저	원격 접속 포트 미개방

원격 접속 포트를 아예 열지 않습니다. 세션 기록이 그 자체로 감사 증거입니다.

8.2 파괴 액션 승인

마켓 정지, 노드 종료, 권한 변경 같은 되돌리기 어려운 조작은 승인 티켓을 요구합니다.

승인은 1인 승인입니다. 티켓을 발급한 운영자와 사용하는 운영자가 같아도 됩니다 (ADR 0019).

종전 판은 서로 달라야 한다고 걸었습니다. 그런데 이 프로젝트의 운영 인원은 1인입니다. 두 인증서를 한 사람이 번갈아 제시하는 것은 담합 방지가 아니라 그 외형일 뿐이었고, 실제로 한 일은 야간 단독 운영 시간대에 정지 명령을 봉쇄한 것이었습니다.

바꾼 뒤에도 남는 방어가 일곱입니다.

- 승인 티켓을 따로 발급하는 단계. 실행 전에 의도를 한 번 더 적습니다
- 티켓 수명 5분
- 한 번 쓰면 소각
- 액션과 대상에 묶음. 다른 대상에 전용할 수 없습니다
- 요청자 묶음. 남의 승인을 가로챌 수 없습니다
- 역할 통제
- 추가만 가능한 감사 기록

사라지는 것은 발급자와 사용자의 분리 한 겹입니다. 단독 운영자의 고의는 사전에 막히지 않고 감사 기록으로만 사후에 드러납니다. 이 대가를 한계에 등재했습니다.

8.3 가드레일

비채택으로 정한 서비스는 계정 권한에서 생성 자체를 막습니다. 문서로만 "안 쓴다"고 적으면 실수로 켜집니다.

9. 비용

9.1 원칙

- 단가는 클라우드 사업자의 공식 가격 파일을 원천으로 씁니다. 요약 사이트 수치를 쓰지 않습니다
- 원천에 없는 값은 유도값 또는 미확인으로 표시하고 그 사실을 표에 남깁니다
- 환산 환율은 1달러 1,400원입니다. 민감도를 함께 적습니다

- 자체 상한은 월 500,000원입니다. 2026-09-08 지도교수 승인으로 종전 100,000원에서 올렸습니다. 승인액은 50만원 이상이고 크레딧으로 지급 예정이며 지급 확인은 아직입니다
- 부가세 10퍼센트를 별도 계상합니다
- 상한은 달력 월 실청구로 판정합니다

9.2 상시면 단가

768시간 기준입니다.

항목	산식	달러
로드밸런서 고정분	0.0225 x 768	17.2800
로드밸런서 변동분	0.008 x 1 x 768	6.1440
애플리케이션 t4g.small 2대	0.0208 x 2 x 768	31.9488
데이터베이스 t4g.small 3대	0.0208 x 3 x 768	47.9232
봇과 관측 t4g.small 1대	0.0208 x 768	15.9744
주소 변환 t4g.nano 2대	0.0052 x 2 x 768	7.9872
공인 주소 2개	0.0050 x 2 x 768	7.6800
블록 스토리지 8노드 20기가	160 x 0.0912 x 768/730	15.3515
외부 전송 20기가 추정	20 x 0.126 x 768/730	2.6512
도메인과 오브젝트 스토리지	정액	0.7200
확정 단가 소계		153.6604

가용영역 간 전송과 세션 매니저 요금은 확인하지 못해 소계에 넣지 않았습니다. 따라서 153.6604는 확정 단가만의 하한입니다.

9.3 달력 월 실청구

달력 월	소계 달러	부가세 포함	원화 환산	상한 500,000원
2026년 9월	76.13	83.74	117,242원	통과. 여유 382,758원
2026년 10월	76.13	83.74	117,242원	통과. 여유 382,758원

세 번째 데이터베이스 노드가 달력 월당 8.95달러를 더합니다. 내역은 인스턴스 7.9872달러와 블록 스토리지 0.9595달러입니다. 봇 노드 등급 상향분 3.99달러도 함께 들어 있습니다. 종전 판은 이 상향분을 빠뜨려 111,096원으로 적었고, 같은 절이 그 값을 상한 판정에 인용하지 말라고 스스로 적어 두었는데 요약이 세 곳에서 인용했습니다.

9.4 환율 민감도

환율	달력 월 실청구	상한 대비
1,350원	113,049원	여유 77.4퍼센트
1,400원	117,242원	여유 76.6퍼센트

1,450원	121,434원	여유 75.7퍼센트
1,500원	125,622원	여유 74.9퍼센트

손익분기 환율은 5,971원입니다. 상한을 올린 뒤로는 환율이 상한 판정을 위협하지 않습니다.

9.5 총계

항목	달러
상시면 확정 단가분	153.66
증명면 창 구동 90시간	7.79
소계	161.45
부가세 10퍼센트	16.15
총계	177.60

원화 환산 248,640원입니다.

지원 범위 안입니다. 기존 크레딧 잔액이 163.71달러이고, 여기에 2026-09-08 지도교수 승인분 50만원 이상이 다음 주에 크레딧으로 지급될 예정입니다. 승인분만으로도 총계를 덮습니다. 지급 확인 전까지는 기존 잔액 163.71달러 기준으로 13.89달러가 부족한 상태이므로, 지급이 늦어지면 그 차액이 현금 지출이 됩니다.

종전 판은 총계를 157.91달러로 적고 잔여 5.80달러가 남는다고 했습니다. 그 계산이 두 곳에서 틀렸습니다. 하나는 확정 단가 소계를 표의 행 합인 153.66이 아니라 135.77로 쓴 것이고, 둘은 정액 항목 0.72달러를 나열만 하고 합산에서 빠뜨린 것입니다. 표의 행 합을 권위로 삼아 다시 냈습니다.

10. 검증

10.1 판정 원칙

사람 눈으로 판정하지 않습니다. 판정은 셋 중 하나입니다.

- SQL 위반 행 수. 0행이면 통과입니다
- 지표 임계. 넘으면 실패입니다
- 종료 코드. 0이 아니면 실패입니다

10.2 무결 판정

복식부기 원장의 불변식 27개를 SQL로 검사합니다. 자산별 합이 0인지, 주문이 중복 접수되지 않았는지, 체결이 중복되지 않았는지, 명령이 유실되지 않았는지를 각각 다른 질의가 봅니다.

파일은 `harness/sql/invariants.sql`입니다. 한 항목이라도 1행을 돌려주면 그 회차는 실패입니다.

27개는 그 파일의 판정 블록 수입니다(2026-09-09 실측). 코드 축은 C01 부터 C25 까지이고, 그중 둘이 각각 두 블록으로 갈라져 블록이 27개가 됩니다. C07 은 주문 표와 저널을 따로 보아 C07 과 C07P 로, C13 은 구현 여부와 저널 기록 여부를 따로 보아 C13a 와 C13b 로 나뉩니다.

애플리케이션이 주기적으로 도는 검증기는 26개입니다(`InvariantCheck` 열거형). 27개에서 C04 하나가 빠집니다. C04 는 지연 제약 트리거가 데이터베이스 안에서 직접 강제하므로 따로 조회할 대상이 아닙니다. 곧 27개가 카탈로그의 크기이고 26개가 앱이 순회하는 크기이며, 둘의 차이는 강제 수단이 다른 한 건뿐입니다.

10.3 장애 주입 카탈로그

주입 스크립트 9종입니다. 각 실험이 무엇을 어떻게 넣는지 적습니다.

실험	주입 대상	원시 연산
E1	데이터베이스 경로 지연	프록시 지연 주입
E2	데이터베이스 연결 절단	프록시 타임아웃 0
E3	캐시 계층 정지	컨테이너 강제 종료
E4	리더 노드 정지	컨테이너 강제 종료
E5	네트워크 분단	네트워크에서 컨테이너 분리
E6	폭주 중 페일오버	봇 부하 상향 후 강제 종료
E7	회색 장애	컨테이너 일시 정지
E8	애플리케이션 내부 교란	주입 도구
E9	커밋 경로 절단	프록시 연결 재설정
E10	커널 계층 지연	미실시. 검증면 전용입니다

E1부터 E9까지는 전부 프록시와 컨테이너 조작입니다. 커널 큐의 실제 거동을 재현하지 않습니다. 그것이 E10의 몫이고 아직 돌리지 않았습니다.

10.4 실측 증거

저장소에 판정 산출물 30건이 있습니다. 2026-09-08 회차가 가장 최근입니다.

증거	무엇을 쟀는가
`replication-fault-20260908`	동기 복제 대상 상실이 쓰기에 미치는 영향과 정족수의 효과
`sync-replication-20260908`	동기 복제 자체의 처리량 대가
`leader-lease-dynamodb-20260908`	임차를 커밋 경로 밖으로 옮긴 효과
`leader-failover-20260908`	두 노드 리더 교체 시간과 세대 번호
`follower-rejoin-20260908`	부하 중 노드 재합류
`pool-db-matrix-20260908-203140`	커넥션 풀과 데이터베이스 규격 교차. 40 OPS
`pool-db-matrix-20260908-204634`	같은 교차. 고부하 시도
`asg-scenario-20260908`	확장과 축소 소요 시간
`wal-fsync-20260907`	저장소 동기화 지연
`ws-admission-20260907`	동시 접속 상한

나머지 20건은 용량 보정과 처리량 측정입니다.

10.5 주요 실측값

항목	값	출처

리더 교체 시간	9.6초에서 13.1초	`leader-failover-20260908`
부하 중 재합류	20.8초	`follower-rejoin-20260908`
동기 복제의 처리량 대가	복제 없음 76.3, 비동기 65.9, 동기 53.0 (초당)	`sync-replication-20260908`
대기 서버 1대 상실 시 접수	초당 97.2에서 0	`replication-fault-20260908`
정족수에서 1대 상실 시 접수	초당 56.0 유지	같음
임차 분리 후 정지 중 갱신	초당 0.35회 지속	`leader-lease-dynamodb-20260908`
데이터베이스 규격 상향의 지연 효과	상위 1퍼센트 60퍼센트 이상 감소	`pool-db-matrix-20260908-*`
확장 소요	컨테이너 생성 1.0초, 앱 기동 15.8초, 투입 1.2초	`asg-scenario-20260908`
주문당 라이터 시간	10.1밀리초. 그중 동기화 6.41밀리초	`wal-fsync-20260907`

10.6 통합 검증

검증 클래스 74개가 전량 통과합니다. 실제 데이터베이스를 띄워 도는 통합 검증이 포함됩니다.

지속적 통합 워크플로 4종이 병합 전에 검증합니다. 백엔드 검증, 웹 검증, 이미지 빌드, 스타일 검사입니다.

10.7 무엇이 아직 검증되지 않았는가

- 배포 대상 인스턴스 실물에서 잰 값이 0건입니다
- 커널 계층 주입이 미실시입니다
- 클라우드 환경에서의 가용영역 상실 재현이 미실시입니다
- 병목 이동(데이터베이스를 키운 뒤 애플리케이션이 병목이 되는 것)이 미확인입니다

11. 일정

11.1 남은 구간

구간	기간	내용
W1	09-09 부터 09-15	클라우드 계정과 네트워크 구성
W2	09-16 부터 09-22	노드 배포와 도메인 연결
W3	09-23 부터 09-29	데이터베이스 이중화와 권한 분리
W4	09-30 부터 10-06	관측 배선과 알람 실투입
W5	10-07 부터 10-09	커널 계층 주입과 병목 이동 실측
W6	10-10 부터 10-16	최종 완주와 발표 준비

10-09에 기능을 동결합니다. 그 뒤에는 실험과 문서만 합니다.

11.2 잘라낼 순서

시간이 모자라면 아래 순서로 뺍니다.

1. 애플리케이션 내부 교란 실험
2. 프록시 지연 실험 재실행
3. 커밋 절단 실험 재실행

무결과 복구 판정에 직접 걸린 실험은 마지막까지 남깁니다.

12. 완료 기준

발표 시점에 아래가 전부 참이어야 합니다.

번호	기준	판정 방법
A1	외부에서 도메인으로 접속됩니다	브라우저 접속 확인
A2	불변식 27개의 위반 행이 0입니다	SQL 실행 결과
A3	리더 교체가 30초 이내에 끝납니다	판정 파일의 시각 차이
A4	세대 번호가 정확히 1 증가하고 구 리더 쓰기가 0입니다	판정 파일
A5	실험 판정 파일이 저장소에 남습니다	파일 존재 확인
B1	2026년 9월과 10월 달력 월 실청구가 각각 500,000원 이하입니다	청구서. 계획값 각 117,242원
B2	장기 접근 키가 0개입니다	계정 설정 확인
C1	병목 계층을 관측으로 특정하고 조작 효과를 실측합니다	조작 전후 지표 비교

C1이 이번에 추가된 기준입니다. 현재 데이터베이스 축은 성립했고 애플리케이션 축이 미확인입니다.

13. 한계

정직하게 적습니다. 여기 적힌 것은 알고 있으면서 해결하지 못한 것입니다.

13.1 측정 평면

번호	한계
1	배포 대상 인스턴스 실물 실측이 0건입니다. 로컬 도커 값이 그대로 성립한다는 보장이 없습니다
2	같은 기계에서 컨테이너 12개에서 27개가 함께 돌았습니다. 성능 수치는 절대값이 아니라 주행 안의 상대 비교입니다
3	가용영역 간 지연을 프록시로 모사했습니다. 실제 지연은 미측정입니다
4	저장소 동기화 지연의 거의 전부가 개발 기계의 파일 시스템

	계층에서 나옵니다. 배포 볼륨에서 다시 재야 합니다

13.2 구조

번호	한계
5	쓰기는 단일 라이터입니다. 노드를 늘려도 쓰기 천장이 오르지 않습니다
6	정족수도 가용영역 동시 상실은 막지 못합니다. 세 노드를 두 가용영역에 나누면 한쪽이 반드시 두 노드를 갖고, 그쪽을 잃으면 남은 한 대가 정족수를 못 채웁니다
7	시계 스큐 내성이 검증 범위 밖입니다. 임차 만료 판정이 노드 시계에 걸립니다
8	봇과 관측이 한 노드에 있습니다. 그 노드가 죽으면 기저 트래픽과 지표 수집이 함께 멈춥니다
9	승격 재생이 엔진 상태를 두 벌로 만듭니다. 그 순간 메모리를 두 배로 씁니다
10	승격 시 동기 복제 대상 재설정이 미배선입니다

13.3 통제

번호	한계
11	단독 운영자의 고의를 사전에 막지 못합니다. 승인을 1인으로 바꾼 대가입니다. 감사 기록으로만 사후에 드러납니다
12	자동 확장 정책이 없습니다. 사람이 자는 동안에는 증설이 일어나지 않습니다

13.4 미배선

번호	한계
13	상시면 컴포즈에 관측 서비스 정의가 0건입니다. 관측은 로컬 카오스 스택에서만 돕니다
14	외부에서 찌르는 프로브가 없습니다. 로드밸런서와 콘텐츠 전송 네트워크 구간의 고장을 잡을 수단이 없습니다
15	알림 수신 주소를 넣지 않았습니다
16	백업 3종이 전부 목표 상태입니다
17	인프라 코드화 파일이 0개입니다

13.5 미확인

번호	한계
18	병목 이동이 미확인입니다. 데이터베이스를 키운 뒤 애플리케

	이션이 병목이 되는 것을 실측하지 못했습니다. 수요를 천장 아래로 고정했기 때문입니다
19	고부하 회차에서 실효 부하가 오르지 않았습니다. 무엇이 상한을 만들었는지 판정하지 못했습니다
20	대기 서버에 읽기를 보내면 커밋이 느려지는지 미측정입니다. 실험 도구는 준비했습니다
21	버스트 가능 인스턴스의 신용 소진이 측정에 미치는 영향이 미확인입니다. 데이터베이스가 상시 0.4에서 0.5코어를 쓰는데 이 등급의 기저 성능이 그 근처입니다
22	리더 임차의 DynamoDB 경로가 기본값이 아닙니다. 2026-09-08 회차에서 그 경로를 재어 87초 정지를 견디는 것을 확인했으나, 배포 기본값은 데이터베이스 쪽이라 상시 구동은 종전 경로로 돕니다. 상시면에서 어느 쪽을 켤지 정하지 않았습니다

13.6 발표 전에 반드시 닫아야 하는 것

13.4의 미배선 다섯 항목 중 관측 배선과 알림 실투입은 완료 기준에 직접 걸립니다. 나머지는 한계로 남긴 채 발표합니다.

부록. 결정 기록

이 문서는 현재 상태의 정본입니다. "그때 왜 그렇게 정했는가"는 `docs/adr/`가 갖습니다. 결정 기록 19편이 있습니다.

이 문서와 결정 기록이 어긋나면 이 문서가 현재 상태의 권위입니다. 다만 결정 기록의 사유를 뒤집으려면 새 결정 기록을 써야 합니다.